

Assignment 04 - Distributed Programming

Problem Analysis

This assignment moves from local concurrency to distributed programming. The implemented mandatory systems are a Smart Home Alarm based on Apache Pekko Cluster and a Tic-Tac-Toe game based on Java RMI.

Exercise 1 - Distributed Smart Home Alarm

Architecture

```
alarm-cluster
node 25520 role=control -> ControlPanelEntity
node 25521 role=keypad -> DistributedKeypadActor
node 25522 role=sensor -> DistributedSensorActor
```

The actors communicate only through actor messages. The control panel registers through Pekko typed Receptionist; keypad and sensor actors subscribe to the service key and send commands/events to the discovered control panel reference. Messages implement `JsonSerializable` and are bound to Pekko Jackson JSON serialization.

State Machine And Safe Recovery

The control entity supports RECOVERY, DISARMED, EXIT_DELAY, ARMED, ENTRY_DELAY, and ALARM.

```
RECOVERY -- valid PIN --> DISARMED
DISARMED -- valid PIN arm --> EXIT_DELAY -- timeout --> ARMED
ARMED -- active-zone sensor --> ENTRY_DELAY -- timeout --> ALARM
ENTRY_DELAY -- valid PIN --> DISARMED
ALARM -- valid PIN --> DISARMED
```

The distributed control entity starts in RECOVERY. It ignores sensors and arm requests until a correct PIN is received. Zone-based partial arming is supported.

Cluster Configuration

`application.conf` enables the cluster provider, Artery remoting, three localhost seed nodes, and Split Brain Resolver with keep-majority.

```
.\mvnw.cmd exec:java "-Dexec.args=control 25520"
.\mvnw.cmd exec:java "-Dexec.args=keypad 25521 demo"
.\mvnw.cmd exec:java "-Dexec.args=sensor 25522 front-door perimeter DOOR demo"
```

Exercise 2 - Distributed Tic-Tac-Toe With Java RMI

The RMI system is based on `TicTacToeHub`, `TicTacToeGame`, and `TicTacToeListener`. The hub is registered in the RMI registry and exports each game as a remote object. Clients can create named games, join existing games, play moves, receive callbacks, and request snapshots.

Game methods are synchronized on the server, preventing conflicting concurrent moves. Callback listeners are kept in a `CopyOnWriteArrayList` and removed on `RemoteException`.

```
.\mvnw.cmd exec:java "-Dexec.mainClass=pcd.ass04.ttt.TicTacToeServer" "-Dexec.args=1099"
.\mvnw.cmd exec:java "-Dexec.mainClass=pcd.ass04.ttt.TicTacToeClient" "-Dexec.args=localhost 1099 create game1 Alice"
.\mvnw.cmd exec:java "-Dexec.mainClass=pcd.ass04.ttt.TicTacToeClient" "-Dexec.args=localhost 1099 join game1 Bob"
```

Verification

Tests cover safe recovery, alarm transitions, zone filtering, RMI game creation/join, legal turns, and winner detection.

```
.\mvnw.cmd test
```

```
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  
BUILD SUCCESS
```

```
.\mvnw.cmd package
```

Packaging completed with BUILD SUCCESS.